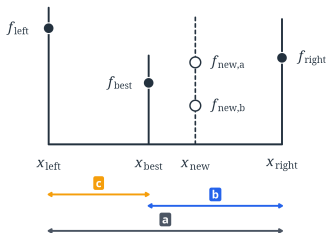
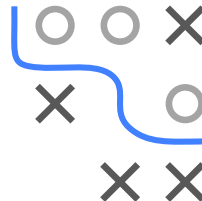


Optimization in Machine Learning

Univariate optimization

Golden section search



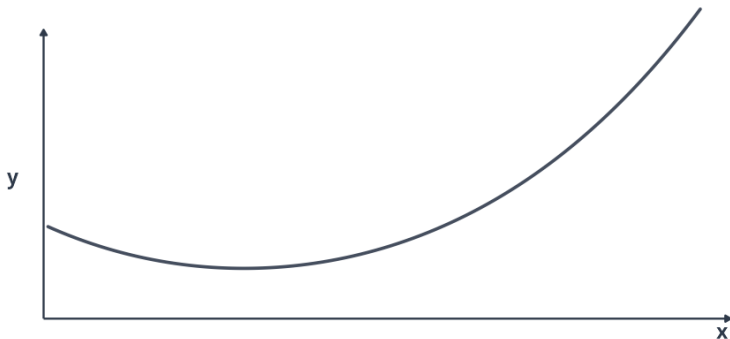
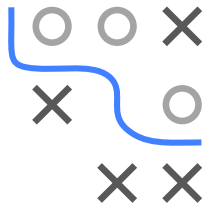
Learning goals

- Simple nesting procedure
- Golden section search

Slides based on [Press et al. 1992](#) (Ch. 10.1)

UNIVARIATE OPTIMIZATION

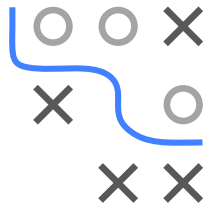
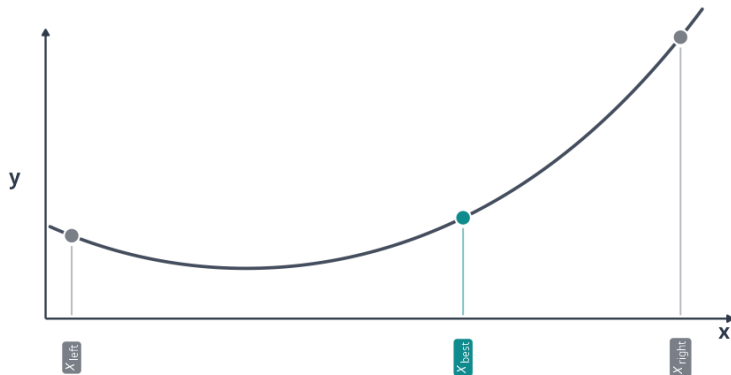
- Setting: optimize univariate $f : \mathbb{R} \rightarrow \mathbb{R}$ over interval $(x_{\text{left}}, x_{\text{right}})$
- Assumption: $-f$ is unimodal and interval $(x_{\text{left}}, x_{\text{right}})$ (called a **bracket**) contains the minimum
- Goal: iteratively shrink the bracket (**bracketing**); gradient-free, works for black-box f



SIMPLE NESTING PROCEDURE

Let $f : \mathbb{R} \rightarrow \mathbb{R}$

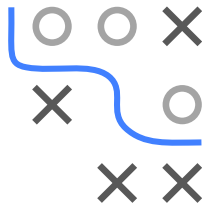
Always maintain three points: left, right, and current best



SIMPLE NESTING PROCEDURE

Let $f : \mathbb{R} \rightarrow \mathbb{R}$

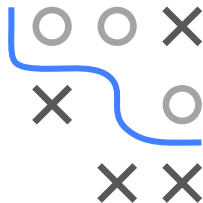
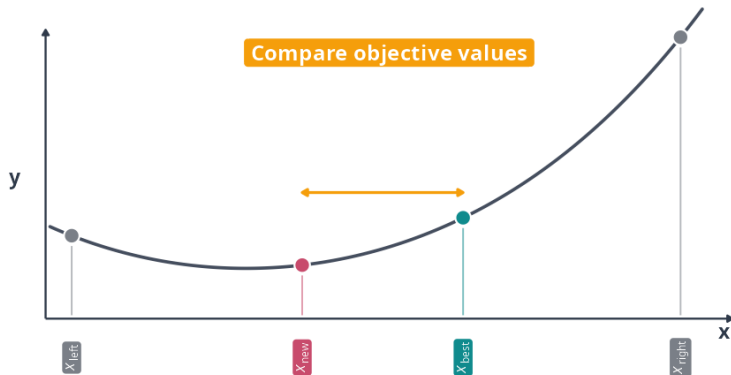
Propose random interval point (def. optimal new proposal later)



SIMPLE NESTING PROCEDURE

Let $f : \mathbb{R} \rightarrow \mathbb{R}$

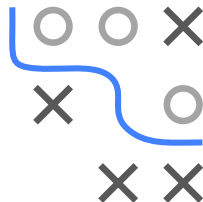
Compare proposal against current best



SIMPLE NESTING PROCEDURE

Let $f : \mathbb{R} \rightarrow \mathbb{R}$

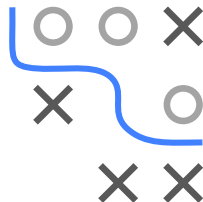
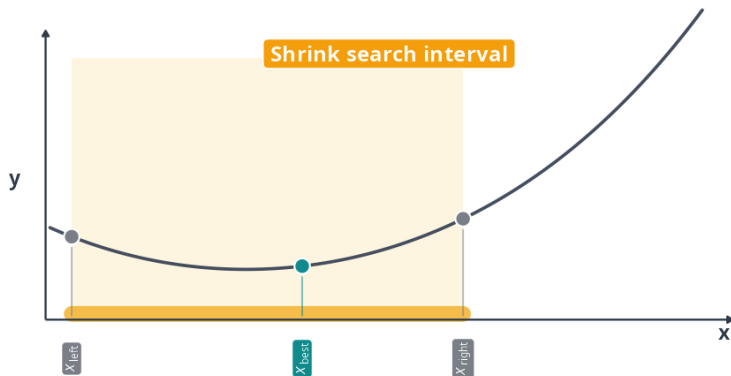
If it is better: proposal becomes current best



SIMPLE NESTING PROCEDURE

Let $f : \mathbb{R} \rightarrow \mathbb{R}$

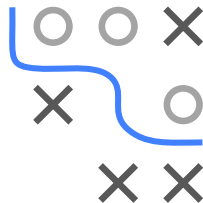
New search interval: around current best



SIMPLE NESTING PROCEDURE

Let $f : \mathbb{R} \rightarrow \mathbb{R}$

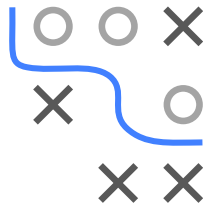
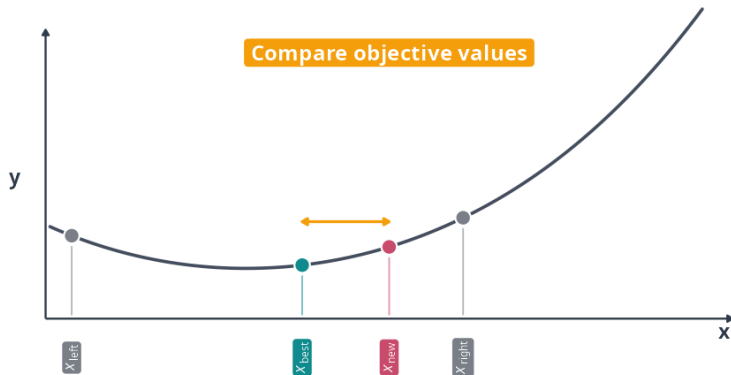
Propose a random point



SIMPLE NESTING PROCEDURE

Let $f : \mathbb{R} \rightarrow \mathbb{R}$

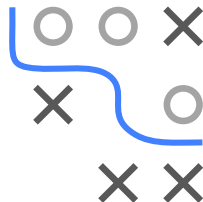
Compare proposal against current best



SIMPLE NESTING PROCEDURE

Let $f : \mathbb{R} \rightarrow \mathbb{R}$

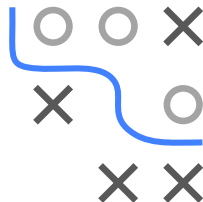
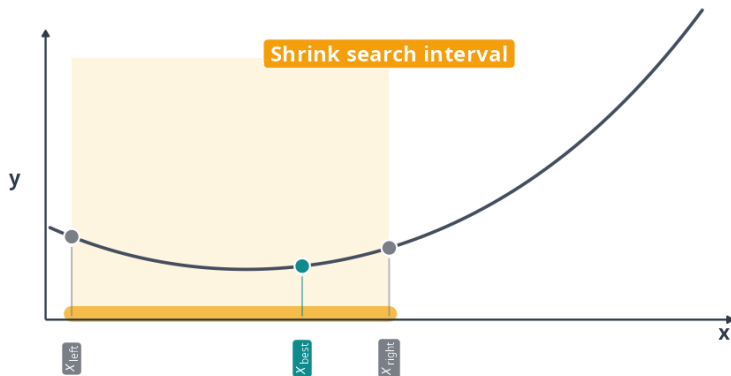
If it is better: proposal becomes current best



SIMPLE NESTING PROCEDURE

Let $f : \mathbb{R} \rightarrow \mathbb{R}$

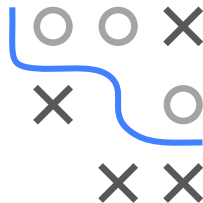
New search interval: around current best



SIMPLE NESTING PROCEDURE

Let $f : \mathbb{R} \rightarrow \mathbb{R}$

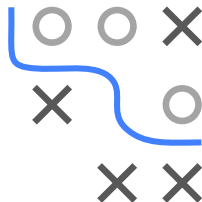
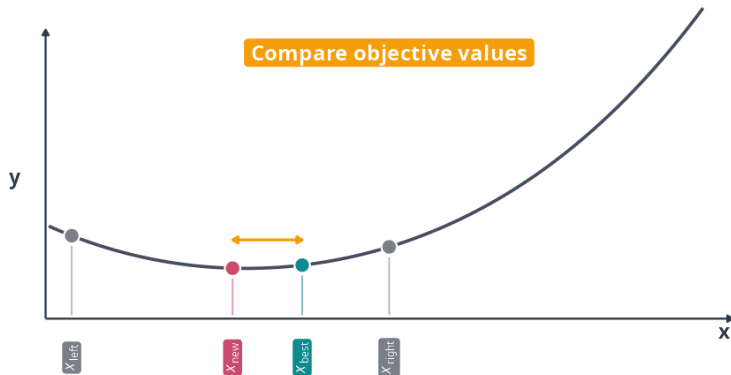
Propose a random point



SIMPLE NESTING PROCEDURE

Let $f : \mathbb{R} \rightarrow \mathbb{R}$

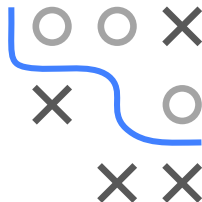
Compare proposal against current best



SIMPLE NESTING PROCEDURE

Let $f : \mathbb{R} \rightarrow \mathbb{R}$

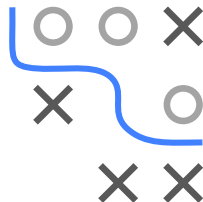
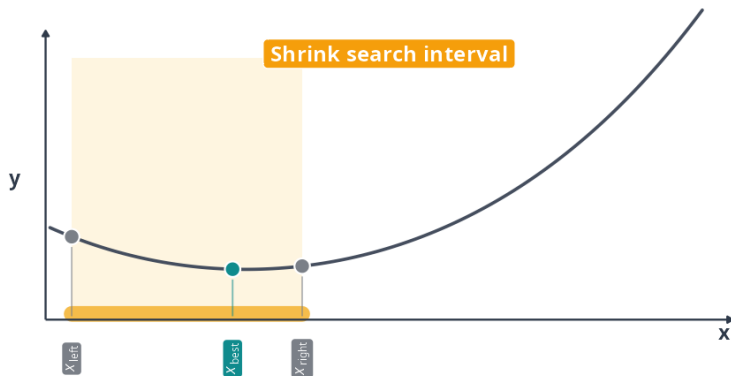
If it is better: proposal becomes current best



SIMPLE NESTING PROCEDURE

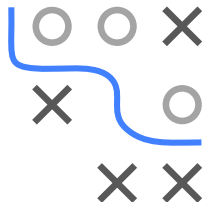
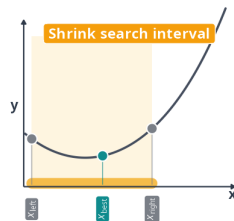
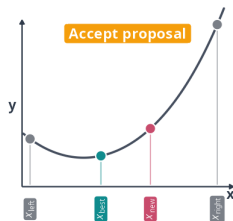
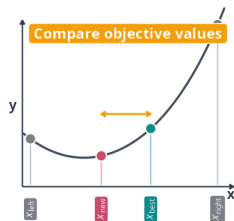
Let $f : \mathbb{R} \rightarrow \mathbb{R}$

New search interval: around current best



SIMPLE NESTING PROCEDURE

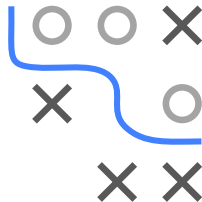
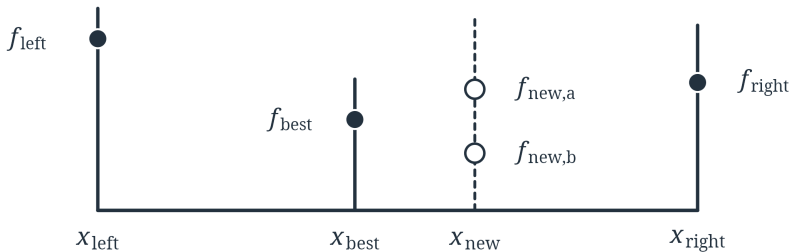
- Initialization: Search interval $(x_{\text{left}}, x_{\text{right}})$, $x_{\text{left}} < x_{\text{right}}$
- Choose x_{best} randomly
- For $t = 0, 1, 2, \dots$
 - Choose x_{new} randomly in $[x_{\text{left}}, x_{\text{right}}]$
 - If $f(x_{\text{new}}) < f(x_{\text{best}})$:
 - $x_{\text{best}} \leftarrow x_{\text{new}}$
 - New interval: points around x_{best}



GOLDEN SECTION SEARCH

Key question: How can x_{new} be chosen better than randomly?

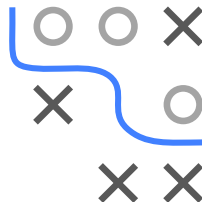
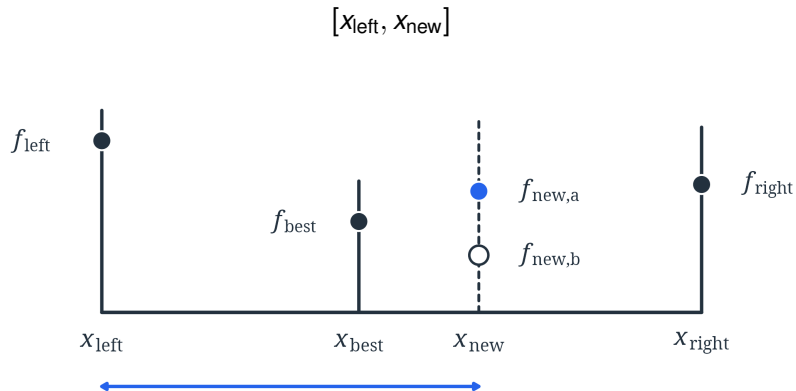
- Principle 1: Always in bigger subinterval to maximize reduction
- Principle 2: x_{new} symmetrically to x_{best} for uniform reduction



Consider two hypothetical outcomes for x_{new} : $f_{\text{new},a}$ and $f_{\text{new},b}$

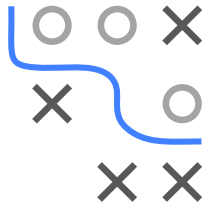
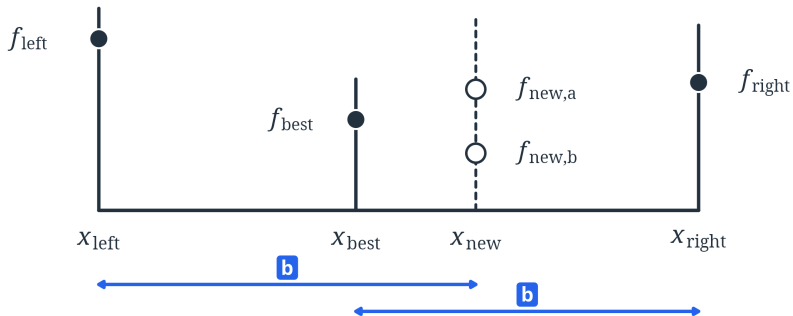
GOLDEN SECTION SEARCH

If outcome is $f_{new,a}$, x_{best} remains best and we search around x_{new} :



GOLDEN SECTION SEARCH

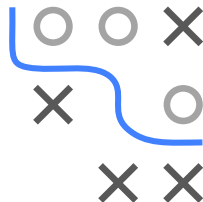
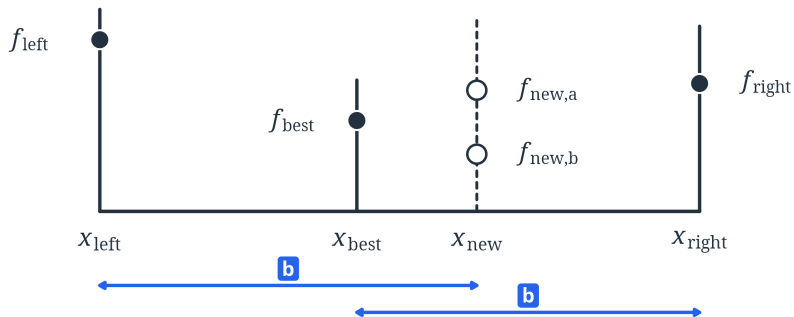
If $f_{new,b}$ is outcome, x_{new} becomes best point and search around x_{new} :

 $[x_{\text{best}}, x_{\text{right}}]$ 

GOLDEN SECTION SEARCH

For uniform reduction the two potential intervals must be equal-sized:

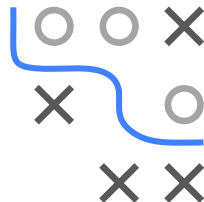
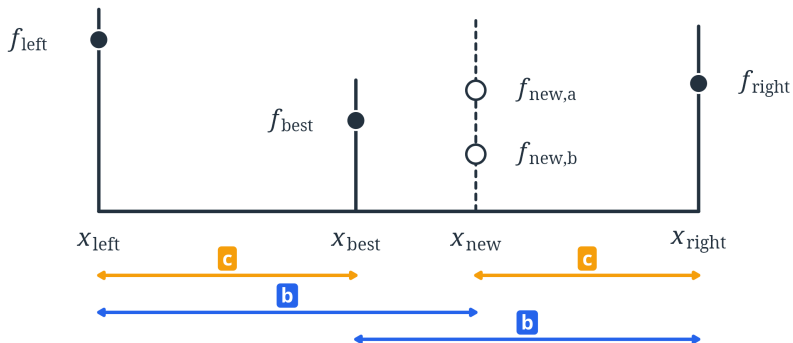
$$b := x_{\text{right}} - x_{\text{best}} = x_{\text{new}} - x_{\text{left}}$$



GOLDEN SECTION SEARCH

One iteration ahead: require again the intervals to be of same size.

$$c := x_{\text{best}} - x_{\text{left}} = x_{\text{right}} - x_{\text{new}}$$



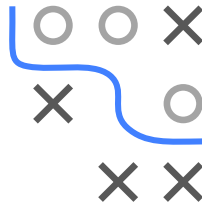
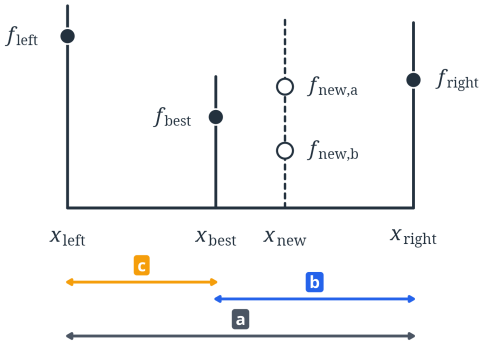
GOLDEN SECTION SEARCH

To summarize, we require:

$$a = x_{\text{right}} - x_{\text{left}}$$

$$b = x_{\text{right}} - x_{\text{best}} = x_{\text{new}} - x_{\text{left}}$$

$$C = X_{\text{best}} - X_{\text{left}} = X_{\text{right}} - X_{\text{new}}$$



GOLDEN SECTION SEARCH

- We require same percentage improvement in each iteration
- For φ reduction factor of interval sizes (a to b , and b to c)

$$\varphi := \frac{b}{a} = \frac{c}{b}$$

$$\varphi^2 = \frac{b}{a} \cdot \frac{c}{b} = \frac{c}{a}$$

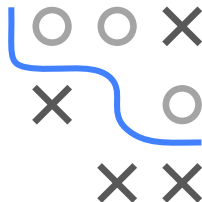
- Divide $a = b + c$ by a :

$$\frac{a}{a} = \frac{b}{a} + \frac{c}{a}$$

$$1 = \varphi + \varphi^2$$

$$0 = \varphi^2 + \varphi - 1$$

- Unique positive solution is $\varphi = \frac{\sqrt{5}-1}{2} \approx 0.618$



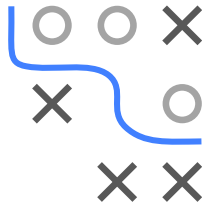
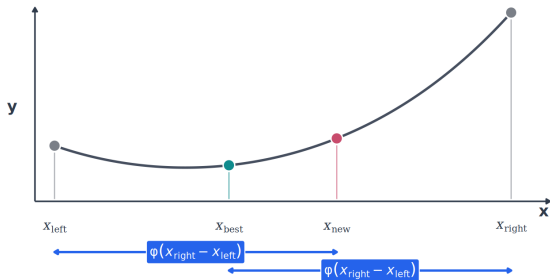
GOLDEN SECTION SEARCH

- Place x_{best} and x_{new} at $\varphi \approx 0.618$ from each endpoint:

$$\begin{aligned}x_{\text{best}} &= x_{\text{right}} - \varphi(x_{\text{right}} - x_{\text{left}}) \\ &= x_{\text{left}} + (1 - \varphi)(x_{\text{right}} - x_{\text{left}})\end{aligned}$$

- By symmetry:

$$\begin{aligned}x_{\text{new}} &= x_{\text{left}} + \varphi(x_{\text{right}} - x_{\text{left}}) \\ &= x_{\text{right}} - (1 - \varphi)(x_{\text{right}} - x_{\text{left}})\end{aligned}$$



GOLDEN SECTION SEARCH — TERMINATION

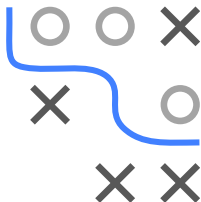
- Simple criterion: stop when bracket width falls below τ :

$$|x_{\text{right}} - x_{\text{left}}| < \tau$$

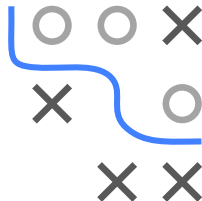
- Issue: τ is absolute — too loose near a minimum close to 0, too tight for a minimum far from 0
- Scale-invariant alternative [Press et al. 1992](#):

$$|x_{\text{right}} - x_{\text{left}}| \leq \tau(|x_{\text{best}}| + |x_{\text{new}}|)$$

where $|x_{\text{best}}| + |x_{\text{new}}|$ serves as a proxy for the scale of the minimum, stopping when the bracket is small relative to the minimum's location



GOLDEN SECTION SEARCH — CONVERGENCE



- Safe method: always converges (given $-f$ is unimodal on $(x_{\text{left}}, x_{\text{right}})$)
- (Slow) linear convergence: bracket shrinks by factor $\varphi \approx 0.618$ per iteration (by construction, see derivation above)